

Practicum No.13**Date:.....****GPU Execution on NVIDIA Jetson Nano****I. Practical Significance**

The experiment using the NVIDIA Jetson Nano demonstrates key concepts of parallel computing by comparing CPU's sequential execution with GPU's parallel processing. Through vector addition, students understand data parallelism, where the same operation is applied simultaneously to large datasets, highlighting the efficiency of GPU architecture with multiple cores. It also introduces memory management concepts such as data transfer between CPU and GPU using CuPy, and enables performance analysis by comparing execution times with NumPy. Overall, the experiment strengthens understanding of GPU computing, memory hierarchy, and real-time embedded processing applications.

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- **Apply** GPU computing principles to optimize computational performance using embedded systems.
(As stated in the Course Structure)
- a) **Identify** suitable tasks for parallel execution using the NVIDIA Jetson Nano (PDO)
- b) **Implement** and compare CPU and GPU computations using NumPy and CuPy (CDO)
- c) **Analyze** execution time and evaluate performance improvement achieved through GPU acceleration (PDO)

III. Relevant CO(s) (As stated in the Course Structure)

- **CO1. Evaluate** GPU system architectures to enhance high-performance computing solutions.

IV. Practical Outcome (PrO) (As stated in the Course Structure)

- **Develop** instruction-level parallelism (ILP) techniques to efficiently perform vector addition for the given set of instructions..

V. Related ADO(s) (As stated in the Course Structure)

- a) **Follow** safe computing and coding practices
- b) **Practice** teamwork and documentation discipline
- c) **Comply** with ethical coding conventions

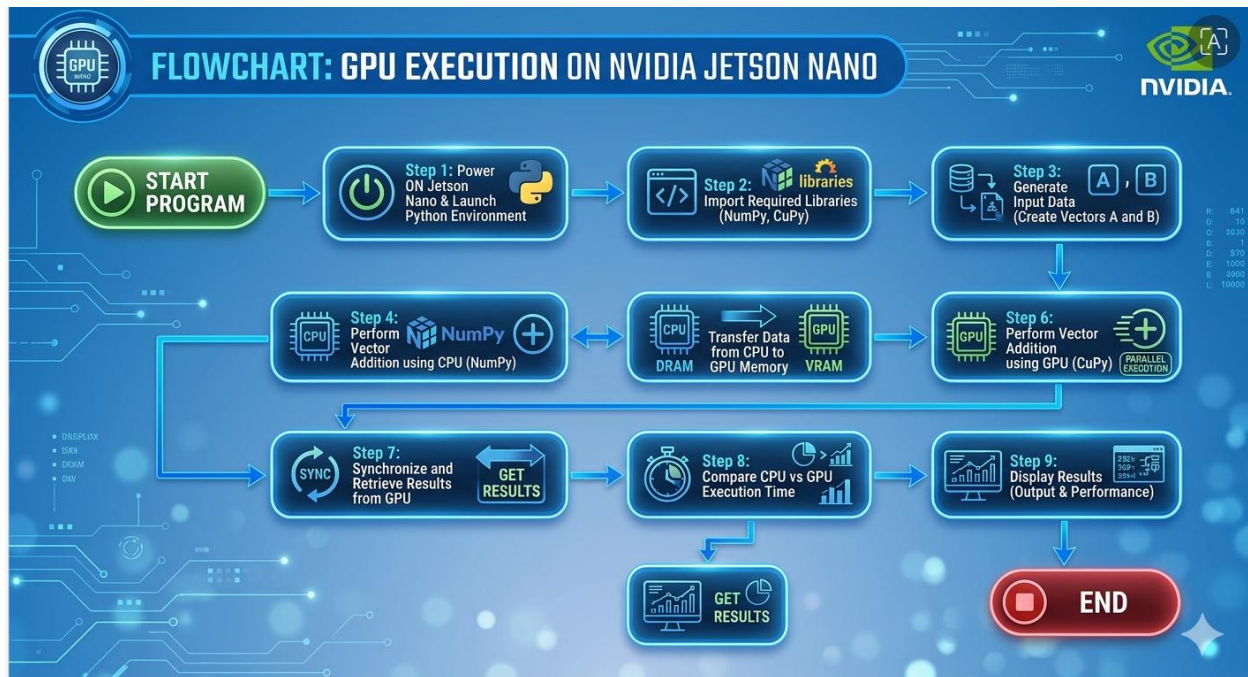
VI. Minimum Underpinning Theory (Include relevant content & images for this practicum.)

GPU execution is based on parallel processing, where multiple computations are performed simultaneously to improve performance. Unlike a CPU, which executes instructions sequentially, a GPU contains many smaller cores that handle thousands of threads in parallel. The NVIDIA Jetson Nano is an embedded platform with an integrated GPU designed for accelerating compute-intensive tasks such as vector operations, image processing, and AI applications.

In GPU computing, tasks are divided into smaller sub-tasks and executed concurrently using a large number of threads. Libraries like CuPy enable GPU-based computation, while NumPy is used for CPU-based execution, allowing performance comparison.

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart (Include probable photos of experimental set-up.)**Students must have:**

- NVIDIA Jetson Nano with power supply and SD card
- JetPack SDK installed (Ubuntu OS with CUDA support)
- Python installed
- Required libraries: NumPy (CPU) and CuPy (GPU)
- Script file: gpu_vector_addition.py
- Input dataset (vector size and random input arrays)
- Basic knowledge of Python programming and parallel computation



VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	NVIDIA Jetson Nano	Quad-core ARM CPU, 4GB RAM, integrated GPU, microSD storage	1 No.
2	Power Supply & Accessories	5V/4A power adapter, HDMI monitor, keyboard, mouse	1 Set.
3	JetPack SDK	Ubuntu OS with CUDA, cuDNN support	1 No.
4	Python	Python 3.x environment	1 No.
5	NumPy	CPU-based numerical computation library	1 No.
6	CuPy	GPU-accelerated array computation library	1 No.
7	Internet Connection	For installation of required libraries and updates	1 No.

IX. Safety Precautions

- Save** work frequently and maintain backup copies of your Python scripts.
- Comment** the code properly and include a header with author name, date, input size, and execution details.
- Close** unnecessary applications on the NVIDIA Jetson Nano to ensure sufficient memory and better GPU performance.
- Validate** input data (vector size, data type) to avoid runtime errors and memory overflow during CPU and GPU execution.

X. Procedure (Self-Instructional)

- Display** the available execution modes to the user (CPU using NumPy and GPU using CuPy).
- Select** the execution mode based on user input (CPU or GPU execution).
- Accept** the required input values such as vector size and input data (arrays A and B).
- Validate** the inputs for correct numeric format and ensure compatibility with the selected execution mode on the NVIDIA Jetson Nano.
- Perform** vector addition using the selected mode (CPU with NumPy or GPU with CuPy).
- Display** the computed output vector and execution time clearly in the console.

- g) **Allow** the user to repeat the execution with different input sizes or modes for comparison.
- h) **Terminate** the program when the user chooses to exit.

Step 1: Boot Jetson Nano

Insert SD card with JetPack SDK

Power ON and complete initial setup

Step 2: Install Required Libraries

Open Terminal and type:

```
sudo apt update
pip3 install numpy
pip3 install cupy-cuda11x
```

Experiment Procedure

Step 3: Create Python Program

Create a file:

```
nano gpu_test.py
```

Paste the code:

```
import numpy as np
import cupy as cp
import time

size = 10**6 # smaller size for Nano

# CPU Data
a_cpu = np.random.rand(size)
b_cpu = np.random.rand(size)

# CPU Execution
start = time.time()
c_cpu = a_cpu + b_cpu
cpu_time = time.time() - start

# GPU Data
a_gpu = cp.array(a_cpu)
b_gpu = cp.array(b_cpu)

# GPU Execution
start = time.time()
c_gpu = a_gpu + b_gpu
cp.cuda.Stream.null.synchronize()
gpu_time = time.time() - start
```

```
print("CPU Time:", cpu_time)
print("GPU Time:", gpu_time)
Step 4: Run the Program
python3 gpu_test.py
```

Sample Output:

Sample Input:

A = [1.2, 2.5, 3.1, 4.0, 5.3]

B = [0.8, 1.5, 2.9, 3.0, 4.7]

Expected Output:

Output Vector (CPU): [2.0, 4.0, 6.0, 7.0, 10.0]

Output Vector (GPU): [2.0, 4.0, 6.0, 7.0, 10.0]

CPU Time: 0.00045

GPU Time: 0.00030

XI. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	NVIDIA Jetson Nano	NVIDIA	Quad-core ARM CPU, 4GB RAM, integrated GPU	1	Used for GPU execution
2.	Power Supply & Accessories	Generic	Generic 5V/4A adapter, HDMI monitor, keyboard, mouse	1 Set	Required for setup
3.	JetPack SDK	NVIDIA	Ubuntu OS with CUDA support	1	Pre-installed
4.	Python with NumPy & CuPy	Open Source	Python 3.x with required libraries	1	Used for implementation

XII. Actual Procedure Followed

1. Booted the NVIDIA Jetson Nano and opened the Python environment.
2. Installed required libraries such as NumPy and CuPy using terminal commands.
3. Created a Python script (gpu_test.py) for vector addition.
4. Generated input vectors and performed vector addition using CPU (NumPy).
5. Transferred data to GPU memory and executed vector addition using GPU (CuPy).
6. Measured and compared execution time for CPU and GPU.
7. Observed the output and verified correctness of results.

XIII. Observations and Dimensional Measurement

The experiment was performed on the NVIDIA Jetson Nano by executing vector addition using CPU with NumPy and GPU with CuPy. It was observed that for smaller input sizes, the execution time of CPU and GPU was nearly similar due to data transfer overhead. However, as the vector size increased, GPU execution time became significantly lower than CPU execution time, demonstrating improved performance through parallel processing. The output vectors from both CPU and GPU were

identical, confirming correctness of computation. Overall, a noticeable speedup was achieved using GPU for larger datasets, with slight variations in execution time due to system load and memory usage.

XIV. Results/Observations

1. The vector addition results obtained using CPU (NumPy) and GPU (CuPy) on the NVIDIA Jetson Nano were identical, confirming correctness of execution.
2. GPU execution showed better performance compared to CPU for larger input sizes, demonstrating the advantage of parallel processing, while for smaller inputs, the difference was minimal due to memory transfer overhead.

XV. Interpretation of Results

The results indicate that GPU-based execution on the NVIDIA Jetson Nano is more efficient for large-scale computations due to its parallel processing capability. The identical outputs from CPU using NumPy and GPU using CuPy confirm the correctness and reliability of GPU computation. The minimal performance difference for smaller inputs highlights the impact of data transfer overhead between CPU and GPU memory. Overall, the experiment demonstrates that GPU acceleration is highly beneficial for data-intensive applications, improving execution speed and computational efficiency.

XVI. Conclusions

The experiment concludes that GPU execution on the NVIDIA Jetson Nano significantly improves performance for large-scale computations through parallel processing. The results confirm that GPU-based computation using CuPy is efficient and reliable, producing correct outputs comparable to CPU execution using NumPy. Hence, GPU acceleration is highly suitable for data-intensive and real-time applications.

XVII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. To perform arithmetic and logical operations using CPU and GPU, and compare their performance.
2. Write a program to perform subtraction and division using CPU (NumPy) and GPU (CuPy), and compare their execution times on the NVIDIA Jetson Nano.
3. Implement bitwise logical operations (AND, OR, XOR) using CPU and GPU, and verify the correctness of results.

XVIII References / Suggestions for further reading

- a) NVIDIA Developer – Jetson Nano Documentation: <https://developer.nvidia.com/embedded/jetson-nano-developer-kit>
- b) CUDA Toolkit Documentation: <https://docs.nvidia.com/cuda/>
CuPy Official Documentation: <https://docs.cupy.dev/en/stable/>
NumPy Official Documentation: <https://numpy.org/doc/>
- c) GPU Computing Basics – YouTube – NVIDIA Developer
- d) Parallel Computing with Python (NumPy vs CuPy) – YouTube Tutorials

XIX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the '*process-related skills/LOs*' (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and '*product-related skills/LOs*'

Note: The weightage for the process-related skills and product-related skills will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correct usage of Python and handling input data	6	
2	Correct implementation of CPU (NumPy) and GPU (CuPy) execution	5	
3	Safe practices, proper coding and documentation	4	
4	Team participation	3	
Product-related Skills: 12 Marks - 40%			
1	Accurate EA results	3	
2	Correct interpretation	2	
3	Quality of outputs (tables/plots)	3	
4	Conclusion quality	2	
5	HOT question responses	1	
6	Journal submission	1	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	Neavis Rishva. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		

